

Quiz: Indexes (1)

Collaboration policy:

You may NOT:

1. discuss the exam with any human other than Mike; this includes:
 - (a) asking your friend for clarification about what a problem is asking
 - (b) asking your friend if they've finished working on the quiz
 - (c) posting questions to github, stackoverflow, or any other online forum

You may:

1. take as much time as needed
2. use any written notes / electronic resources you would like (including postgres documentation and AI systems like ChatGPT)
3. use the lambda server and connect to a running postgres instance
4. ask Mike to clarify questions via email

Name:

All of the problems assume the following schema.

```
CREATE TABLE users (  
    id_users BIGINT PRIMARY KEY,  
    created_at TIMESTAMPTZ,  
    username TEXT UNIQUE NOT NULL  
);  
  
CREATE TABLE tweets (  
    id_tweets BIGINT UNIQUE NOT NULL,  
    id_users BIGINT REFERENCES users(id_users),  
    in_reply_to_user_id BIGINT REFERENCES users(id_users),  
    created_at TIMESTAMPTZ,  
    text TEXT  
);  
  
CREATE TABLE tweet_tags (  
    id_tweets BIGINT REFERENCES tweets(id_tweets),  
    tag TEXT,  
    PRIMARY KEY(id_tweets, tag)  
);
```

1. Recall that certain constraints create indexes on the appropriate columns. List the equivalent `CREATE INDEX` commands that are run by these constraints.

Solution:

```
CREATE UNIQUE INDEX users(id_users);  
CREATE UNIQUE INDEX users(username);  
CREATE UNIQUE INDEX tweets(id_tweets);  
CREATE UNIQUE INDEX tweet_tags(id_tweets, tag);
```

Missing any of the indexes or adding any not appearing here resulted in -2 penalty. In particular, the `REFERENCES` constraint never creates an index. So the following indexes are wrong.

```
CREATE INDEX tweets(id_users);  
CREATE INDEX tweets(in_reply_to_user_id);  
CREATE INDEX tweet_tags(id_tweets);
```

Note: When solving all subsequent problems, assume only the indexes above have been created. For example, if problem 3 asks you to create an index, you should not assume that index has been created when working on problem 4.

2. List the scan methods applicable for the following SQL query.

```
SELECT count(*)
FROM users
WHERE id_users >= :min_id_users
      AND username >= :min_username
      AND username < :max_username
```

Solution: Sequential Scan, Bitmap Scan, Index Scan

If index scan is missing, the penalty is only -1 because it is unlikely to be efficient.

3. Create index(es) so that the following query can use an index only scan.

Do not create any unneeded indexes; if no new indexes are needed, say so.

```
SELECT count(*)
FROM users
WHERE lower(username)=:username;
```

Solution:

```
CREATE INDEX ON users(lower(username));
```

It is important that the function call to `lower` be present in the index. If it is not, -2 points.

4. Create index(es) so that the following query can use an index only scan, avoid an explicit sort, and take advantage of the LIMIT clause for faster processing.

Do not create any unneeded indexes; if no new indexes are needed, say so.

```
SELECT id_users
FROM users
WHERE created_at <=: created_at
ORDER BY created_at DESC
LIMIT 10;
```

Solution:

```
CREATE INDEX ON users(created_at, id_users);
```

The `id_users` column is necessary for the index to be covering and allow an index only scan. The `ORDER BY` clause does not require an explicit sort because it is ordering by the column in the index. It does not matter whether we order by `DESC` or `ASC`. There would be no way to efficiently do both the `ORDER BY` and `WHERE` clause if they used different columns.

You got -1 point if you wrote:

```
CREATE INDEX ON users(created_at) INCLUDE (id_users);
```

The `INCLUDE` syntax is intended for `UNIQUE` indexes only to indicate that there is a `UNIQUE` constraint on some columns but not others.

5. Create index(es) so that the following query will run as efficiently as possible.

Do not create any unneeded indexes; if no new indexes are needed, say so.

```
SELECT id_users, username
FROM users
JOIN tweets USING (id_users)
WHERE in_reply_to_user_id = :in_reply_to_user_id
LIMIT 10;
```

Solution:

```
CREATE INDEX ON tweets(in_reply_to_user_id, id_users);
CREATE INDEX ON users(id_users, username);
```

The second index is needed to allow an index only scan on the users table. Whether the second index is actually used or not will depend on the structure of the data. With the index, we can do an index only scan; but the index has a much lower fanout due to the additional `username` column. Depending on the length of usernames (which affects the number of tuples per index page and hence fanout), it may be faster to do an index scan on the original index. In either case, the difference will be only a small number of page accesses.

You got -1 point if you did not include the second index.

6. Create index(es) so that the following query will run as efficiently as possible.

Do not create any unneeded indexes; if no new indexes are needed, say so.

```
SELECT username, count(*)
FROM users
JOIN tweets USING (id_users)
WHERE in_reply_to_user_id = :in_reply_to_user_id
GROUP BY username;
```

Solution:

```
CREATE INDEX ON tweets(id_users, username);
CREATE INDEX ON tweets(in_reply_to_user_id, id_users);
```

Note that postgres cannot use a GroupAggregate to implement the GROUP BY here without an explicit sort.

7. Consider the following SQL query.

```
SELECT tag,count(*) AS count
FROM tweet_tags
GROUP BY tag
ORDER BY count DESC
LIMIT 10;
```

- a) Create index(es) so that the query will run as efficiently as possible.
Do not create any unneeded indexes; if no new indexes are needed, say so.

Solution:

```
CREATE INDEX ON tweet_tags(tag);
```

The previous index on `tweet_tags(id_tweets, tag)` has the columns in the wrong order.

- b) Given the indexes created above, is the LIMIT clause likely to significantly speed up the query? Explain why/why not.

Solution: No.

The index above means that an index only scan can be used. This will return rows in sorted order, which allows a GroupAggregate to be used. Both of these strategies return data one at a time allow the results to be returned in sorted order. The ORDER BY clause, however, will require an explicit sort because it is on a different column. (In general, count operations cannot be sped up with an index.)

If you said **Yes** you could also get full credit if you explained that the LIMIT clause will be implemented by using a top-N heap sort in the ORDER BY clause. This reduces the runtime from $\Theta(n \log n)$ to $\Theta(n + N \log N)$.

The runtime above is still linear in n , which is relatively slow. I think No is the better answer because a “significant” speed up implies a sublinear runtime.

You got -1 if you did not discuss the interaction of the GROUP BY and ORDER BY clauses.

You got -1 if you did not explicitly name the strategies used.

For example, your explanation should not be equally valid for the query

```
SELECT tag,count(*) AS count
FROM tweet_tags
GROUP BY tag
ORDER BY tag DESC
LIMIT 10;
```

